```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import zipfile
import urllib.request
import os

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer

from sklearn.naive_bayes import MultinomialNB
from sklearn.linear_model import LogisticRegression
from sklearn.svm import LinearSVC

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    balanced_accuracy_score,
    matthews_corrcoef,
    average_precision_score,
    confusion_matrix,
    classification_report,
    precision_recall_curve
)

print("Libraries imported successfully!")
```

```
Libraries imported successfully!
```

```
# Dataset URL
url = (
    "https://archive.ics.uci.edu/static/public/"
    "228/sms+spam+collection.zip"
)

zip_path = "sms_spam_collection.zip"
extract_folder = "sms_dataset"

# Download the dataset
if not os.path.exists(zip_path):
    urllib.request.urlretrieve(url, zip_path)

# Extract the dataset
with zipfile.ZipFile(zip_path, "r") as zip_ref:
    zip_ref.extractall(extract_folder)

# Find the SMS data file
data_path = os.path.join(
    extract_folder,
    "SMSSpamCollection"
)

# Load the dataset
df = pd.read_csv(
    data_path,
    sep="\t",
    header=None,
    names=["label", "message"],
    encoding="utf-8"
)

print("Dataset loaded successfully!")
print(df.head())

print("\nDataset shape:", df.shape)
print("\nClass distribution:")
print(df["label"].value_counts())
```

```
Dataset loaded successfully!
  label                                            message
0   ham  Go until jurong point, crazy.. Available only ...
1   ham                      Ok lar... Joking wif u oni...
```

```
2  spam  Free entry in 2 a wkly comp to win FA Cup fina...
3   ham  U dun say so early hor... U c already then say...
4   ham  Nah I don't think he goes to usf, he lives aro...

Dataset shape: (5572, 2)

Class distribution:
label
ham     4825
spam     747
Name: count, dtype: int64
```

```
# Count messages in each class
class_counts = df["label"].value_counts()

print("Number of messages in each class:")
print(class_counts)

# Calculate class percentages
class_percentages = (
    df["label"].value_counts(normalize=True) * 100
)

print("\nClass percentages:")
print(class_percentages.round(2))

# Plot class distribution
plt.figure(figsize=(7, 5))

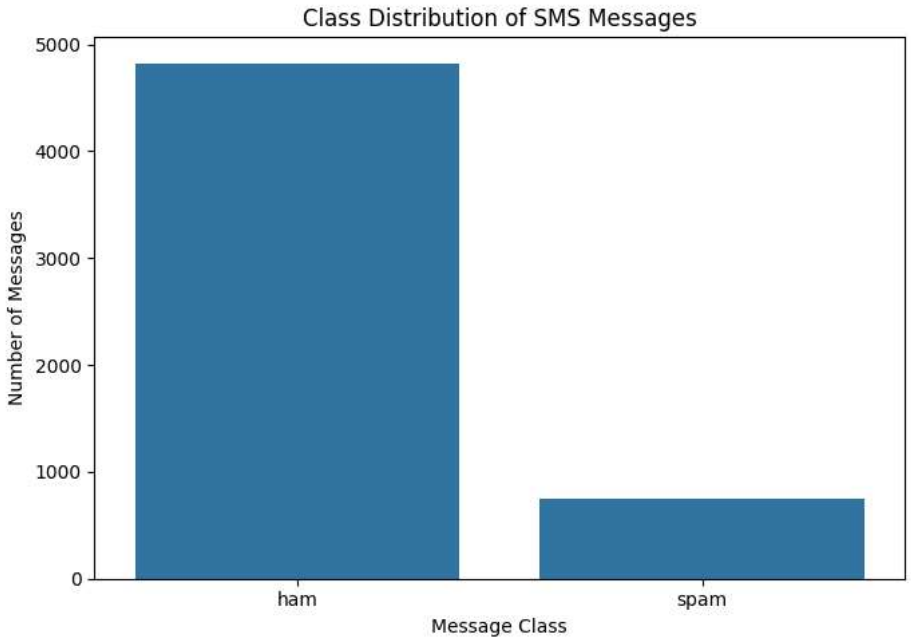
sns.countplot(
    data=df,
    x="label"
)

plt.title("Class Distribution of SMS Messages")
plt.xlabel("Message Class")
plt.ylabel("Number of Messages")

plt.tight_layout()
plt.show()
```

```
Number of messages in each class:
label
ham     4825
spam     747
Name: count, dtype: int64

Class percentages:
label
ham     86.59
spam    13.41
Name: proportion, dtype: float64
```



```
# Remove missing values
df = df.dropna(subset=["label", "message"])

# Remove duplicate messages for this experiment
df = df.drop_duplicates(
    subset=["label", "message"]
).reset_index(drop=True)

# Normalize class labels
df["label"] = df["label"].str.lower().str.strip()

# Keep valid labels only
df = df[df["label"].isin(["ham", "spam"])]

# Convert labels into binary values
# ham = 0, spam = 1
df["target"] = df["label"].map({
    "ham": 0,
    "spam": 1
})

X = df["message"]
y = df["target"]

print("Preprocessing completed!")
print("Total messages:", len(df))
print("\nLabel distribution:")
print(y.value_counts())
```

```
Preprocessing completed!
Total messages: 5169

Label distribution:
target
0    4516
1     653
Name: count, dtype: int64
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))

print("\nTraining class distribution:")
print(y_train.value_counts(normalize=True).round(3))

print("\nTesting class distribution:")
print(y_test.value_counts(normalize=True).round(3))
```

```
Training samples: 4135
Testing samples: 1034

Training class distribution:
target
0    0.874
1    0.126
Name: proportion, dtype: float64

Testing class distribution:
target
0    0.873
1    0.127
Name: proportion, dtype: float64
```

```
Training samples: 4135
Testing samples: 1034

Training class distribution:
target
0    0.874
1    0.126
```

```
Name: proportion, dtype: float64

Testing class distribution:
target
0    0.873
1    0.127
Name: proportion, dtype: float64
```

```
TF-IDF conversion completed!
Training feature shape: (4135, 10000)
Testing feature shape: (1034, 10000)
```

```
models = {
    "Naive Bayes": MultinomialNB(),

    "Logistic Regression": LogisticRegression(
        max_iter=2000,
        random_state=42
    ),

    "Linear SVM": LinearSVC(
        random_state=42
    )
}

# Store trained models and predictions
results = {}
predictions = {}
scores = {}

for name, model in models.items():

    # Train model
    model.fit(X_train_tfidf, y_train)

    # Predict test labels
    y_pred = model.predict(X_test_tfidf)

    # Obtain scores for PR-AUC
    if hasattr(model, "predict_proba"):
        y_score = model.predict_proba(
            X_test_tfidf
        )[:, 1]
    else:
        y_score = model.decision_function(
            X_test_tfidf
        )

    predictions[name] = y_pred
    scores[name] = y_score

    print(f"{name} training completed!")

print("\nAll models trained successfully!")
```

```
Naive Bayes training completed!
Logistic Regression training completed!
Linear SVM training completed!

All models trained successfully!
```

```
evaluation_results = []

for name in models:

    y_pred = predictions[name]
    y_score = scores[name]

    metrics = {
        "Model": name,

        "Accuracy": accuracy_score(
            y_test, y_pred
        ),

        "Precision": precision_score(
```

```
            y_test, y_pred,
            zero_division=0
        ),

        "Recall": recall_score(
            y_test, y_pred,
            zero_division=0
        ),

        "F1-Score": f1_score(
            y_test, y_pred,
            zero_division=0
        ),

        "Balanced Accuracy": balanced_accuracy_score(
            y_test, y_pred
        ),

        "MCC": matthews_corrcoef(
            y_test, y_pred
        ),

        "PR-AUC (Average Precision)": average_precision_score(
            y_test, y_score
        )
    }

    evaluation_results.append(metrics)

# Create comparison DataFrame
results_df = pd.DataFrame(evaluation_results)

# Display results
print("\nMODEL EVALUATION RESULTS\n")

print(
    results_df.round(4).to_string(index=False)
)
```

```
MODEL EVALUATION RESULTS

              Model  Accuracy  Precision  Recall  F1-Score  Balanced Accuracy    MCC  PR-AUC (Average Precision)
        Naive Bayes    0.9652     1.0000  0.7252    0.8407             0.8626 0.8351                      0.9505
Logistic Regression    0.9555     0.9775  0.6641    0.7909             0.8310 0.7850                      0.9624
         Linear SVM    0.9787     0.9580  0.8702    0.9120             0.9323 0.9013                      0.9765
```

```
for name in models:

    print("\n" + "=" * 55)
    print("MODEL:", name)
    print("=" * 55)

    print(
        classification_report(
            y_test,
            predictions[name],
            labels=[0, 1],
            target_names=["Ham", "Spam"],
            zero_division=0,
            digits=4
        )
    )
```

```
=======================================================
MODEL: Naive Bayes
=======================================================
              precision    recall  f1-score   support

         Ham     0.9617    1.0000    0.9805       903
        Spam     1.0000    0.7252    0.8407       131

    accuracy                         0.9652      1034
   macro avg     0.9808    0.8626    0.9106      1034
weighted avg     0.9665    0.9652    0.9628      1034
```

```
======================================================
MODEL: Logistic Regression
======================================================
              precision    recall  f1-score   support

         Ham     0.9534    0.9978    0.9751       903
        Spam     0.9775    0.6641    0.7909       131

    accuracy                         0.9555      1034
   macro avg     0.9655    0.8310    0.8830      1034
weighted avg     0.9565    0.9555    0.9518      1034


======================================================
MODEL: Linear SVM
======================================================
              precision    recall  f1-score   support

         Ham     0.9814    0.9945    0.9879       903
        Spam     0.9580    0.8702    0.9120       131

    accuracy                         0.9787      1034
   macro avg     0.9697    0.9323    0.9499      1034
weighted avg     0.9785    0.9787    0.9783      1034
```

```
metrics_to_plot = [
    "Accuracy",
    "Precision",
    "Recall",
    "F1-Score",
    "Balanced Accuracy"
]

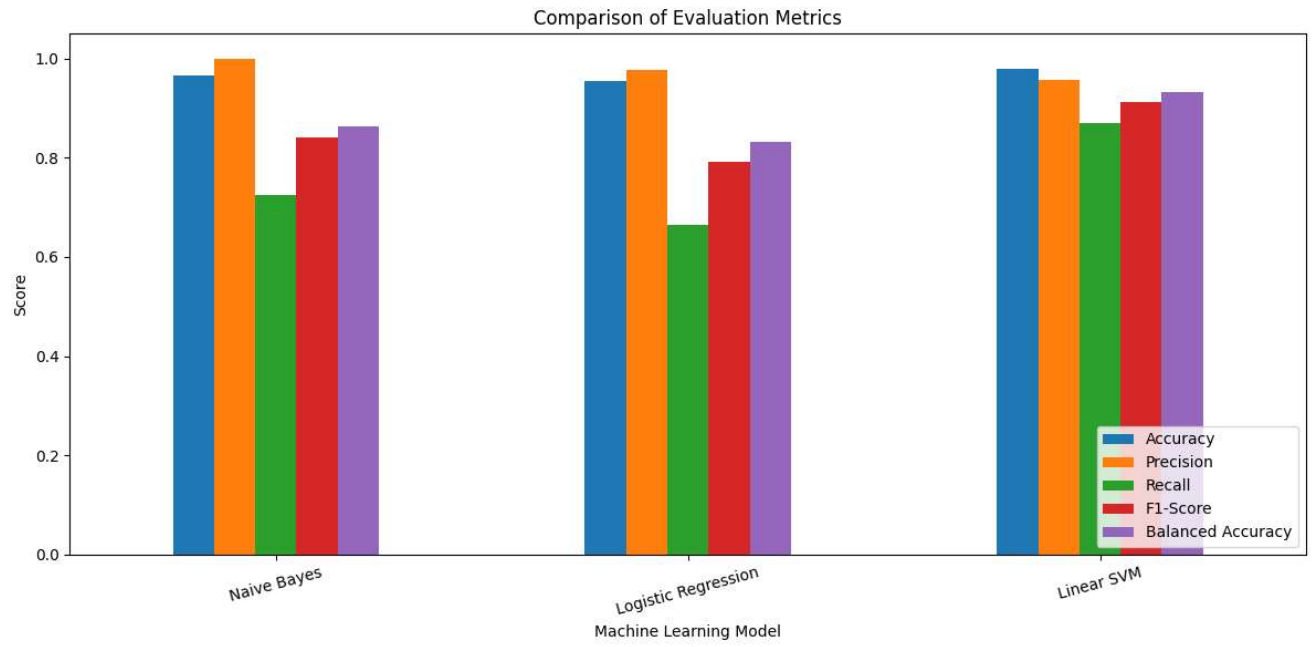
plot_data = results_df.set_index("Model")[
    metrics_to_plot
]

plot_data.plot(
    kind="bar",
    figsize=(12, 6)
)

plt.title("Comparison of Evaluation Metrics")
plt.xlabel("Machine Learning Model")
plt.ylabel("Score")
plt.ylim(0, 1.05)

plt.xticks(rotation=15)
plt.legend(loc="lower right")

plt.tight_layout()
plt.show()
```




```
fig, axes = plt.subplots(
    1, 3,
    figsize=(16, 4)
)

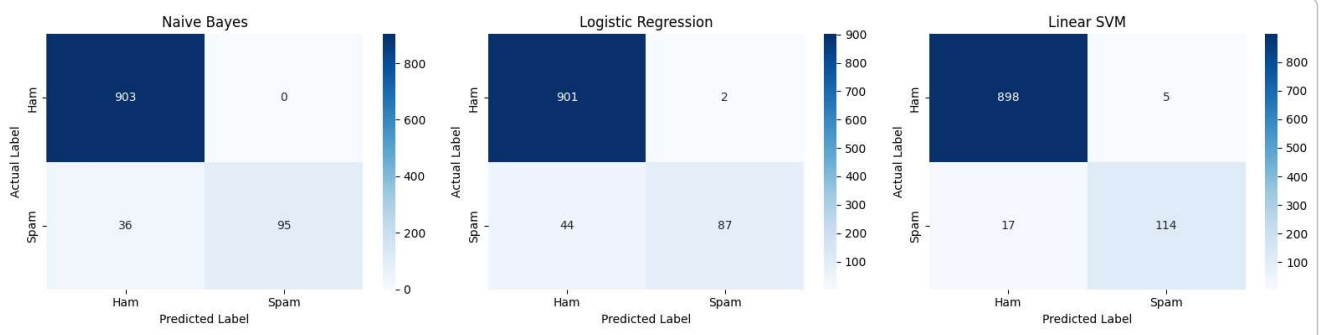
for ax, name in zip(axes, models):

    cm = confusion_matrix(
        y_test,
        predictions[name],
        labels=[0, 1]
    )

    sns.heatmap(
        cm,
        annot=True,
        fmt="d",
        cmap="Blues",
        xticklabels=["Ham", "Spam"],
        yticklabels=["Ham", "Spam"],
        ax=ax
    )

    ax.set_title(name)
    ax.set_xlabel("Predicted Label")
    ax.set_ylabel("Actual Label")

plt.tight_layout()
plt.show()
```




```
plt.figure(figsize=(9, 6))

for name in models:

    precision_values, recall_values, _ = (
        precision_recall_curve(
            y_test,
            scores[name]
        )
    )

    ap = average_precision_score(
        y_test,
        scores[name]
    )

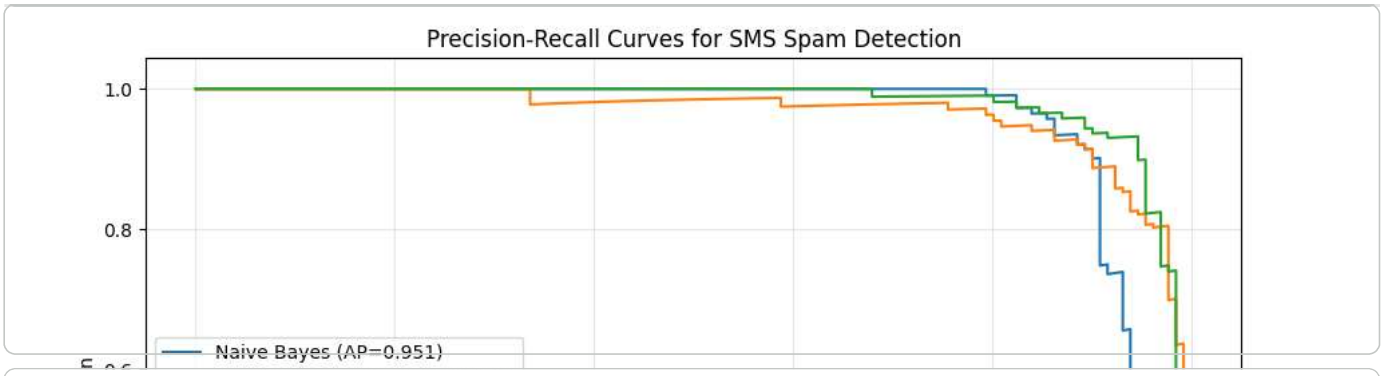
    plt.plot(
        recall_values,
        precision_values,
        label=f"{name} (AP={ap:.3f})"
    )

# Baseline equals the proportion of positive samples
positive_rate = y_test.mean()

plt.axhline(
    y=positive_rate,
    linestyle="--",
    label=f"Positive class prevalence={positive_rate:.3f}"
)

plt.title("Precision-Recall Curves for SMS Spam Detection")
plt.xlabel("Recall")
plt.ylabel("Precision")
plt.legend()
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```




```
metric_columns = [
    "Accuracy",
    "Precision",
    "Recall",
    "F1-Score",
    "Balanced Accuracy",
    "MCC",
    "PR-AUC (Average Precision)"
]

print("\nBEST MODEL FOR EACH METRIC\n")

for metric in metric_columns:

    best_index = results_df[metric].idxmax()

    best_model = results_df.loc[
        best_index, "Model"
```

Could not connect to the reCAPTCHA service. Please check your internet connection and reload to get a reCAPTCHA challenge.